

Orbital Transfer Calculator

Design Notes and Mathematical Background

Yejun (Henry) Kim

October 10, 2026

Contents

1	Introduction and Goals	1
2	Relevant Equations	2
2.1	Conventions	2
2.2	Hohmann transfer	2
2.3	Bi-elliptic transfer	3
2.4	Elliptic transfer on a common apse line	3
2.5	Single-impulse transfer with a rotated apse line	3
2.6	Two-impulse plane-change transfer	3
3	Design Decisions	6
3.1	Free time and the p -parametrization instead of Lambert	6
3.2	Removing Pyodide and porting to native JavaScript	6
3.3	Choice of optimizer: Nelder–Mead versus CMA-ES versus Lipschitz branch and bound	8
3.4	Safeguards in the solver	10
3.5	Known limitations and future work	10
3.6	Code organization	11
4	Features and Images	11
4.1	Feature list	11
4.2	Screenshots	11
4.3	Rendered trajectories	13
4.4	Verification summary	14

1 Introduction and Goals

Following my class in Astronautics, I was interested in improving my understanding of the various orbital maneuvers. From memory, we had learned about the Hohmann Transfer, Bi-Elliptic Hohmann Transfer, Common Apse Line, and Apse Line Rotation maneuvers. We had also mentioned Plane Change maneuvers, but we hadn't gone into it in detail. I began this project primarily because I was interested in exploring the plane change maneuver further, especially because it was explicitly outlined as a challenge in Lecture 37 - Advanced Orbital Maneuvering. The implementation of the first four orbital transfers was pretty straightforward, but the last one proved to be a challenge.

The Orbital Transfer Calculator is a browser-based tool that computes and visualizes the orbital transfers taught in an introductory astronautics course: Hohmann, bi-elliptic, common-apse-line, apse-line-rotation, and the two-impulse *minimum- Δv plane-change* transfer. The user enters a central body (mass and radius) and the orbit parameters; the application reports the burns, transfer time and geometry and draws the initial orbit, target orbit and transfer arc in 3-D with Desmos [21]. Source: [22].

The project had four goals.

1. **Correctness.** Every closed-form transfer is checked against an independent vis-viva reference; the plane-change solver is checked against closed-form special cases and against an independent high-effort search.
2. **Hardest case done properly.** Finding the cheapest two-impulse transfer between two arbitrary (inclined, eccentric) orbits is a non-convex optimization problem. The goal was a solver that is global in practice, fast enough to be interactive, and that reports which basin it found.
3. **Zero-install and static hosting.** Everything runs client-side in plain JavaScript, with no server, no build step and no runtime download beyond the Desmos script.
4. **Maintainable structure.** Pure math modules with no DOM access (unit-tested in Node), one schema per transfer type driving the input form, and one plotter per transfer type.

The plane-change solver was prototyped in Python (NumPy) and run in the browser through Pyodide. It was later ported to native JavaScript (Section 3.2); the Python code is retained as an independent reference implementation and as the generator of test fixtures.

2 Relevant Equations

2.1 Conventions

All lengths are in km, times in s, μ in km^3/s^2 internally (following standard treatments [6, 7]; the UI supplies $\mu = GM$ in m^3/s^2 , converted by 10^{-9}). An orbit is given by apoapsis and periapsis radii r_a, r_p and, in 3-D, inclination i , right ascension of the ascending node Ω and argument of periapsis ω :

$$a = \frac{r_a + r_p}{2}, \quad e = \frac{r_a - r_p}{r_a + r_p}, \quad p = a(1 - e^2), \quad r(\nu) = \frac{p}{1 + e \cos \nu}. \quad (1)$$

Position and velocity in the perifocal frame, and the rotation to the inertial frame, are

$$\mathbf{r}_{pf} = r \begin{bmatrix} \cos \nu \\ \sin \nu \\ 0 \end{bmatrix}, \quad \mathbf{v}_{pf} = \sqrt{\frac{\mu}{p}} \begin{bmatrix} -\sin \nu \\ e + \cos \nu \\ 0 \end{bmatrix}, \quad \mathbf{Q} = R_z(\Omega)R_x(i)R_z(\omega). \quad (2)$$

The orbit normal is $\hat{\mathbf{n}} = \mathbf{Q}[0, 0, 1]^T$. Vis-viva: $v^2 = \mu(2/r - 1/a)$.

2.2 Hohmann transfer

The classical two-impulse transfer [9]. For circular coplanar orbits of radii $r_1 < r_2$ and transfer semi-major axis $a_t = (r_1 + r_2)/2$:

$$\Delta v_1 = \sqrt{\frac{\mu}{r_1}} \left(\sqrt{\frac{2r_2}{r_1 + r_2}} - 1 \right), \quad \Delta v_2 = \sqrt{\frac{\mu}{r_2}} \left(1 - \sqrt{\frac{2r_1}{r_1 + r_2}} \right), \quad t = \pi \sqrt{a_t^3 / \mu}. \quad (3)$$

2.3 Bi-elliptic transfer

With an intermediate apoapsis radius r_i , two half-ellipses of semi-major axes $a_{1i} = (r_1 + r_i)/2$ and $a_{i2} = (r_i + r_2)/2$ are used. Using angular momenta $h_{1i} = \sqrt{2\mu r_1 r_i / (r_1 + r_i)}$ and $h_{i2} = \sqrt{2\mu r_i r_2 / (r_i + r_2)}$,

$$\Delta v_1 = \left| \frac{h_{1i}}{r_1} - \sqrt{\frac{\mu}{r_1}} \right|, \quad \Delta v_2 = \left| \frac{h_{i2}}{r_i} - \frac{h_{1i}}{r_i} \right|, \quad \Delta v_3 = \left| \sqrt{\frac{\mu}{r_2}} - \frac{h_{i2}}{r_2} \right|, \quad (4)$$

$$t = \pi \left(\sqrt{a_{1i}^3 / \mu} + \sqrt{a_{i2}^3 / \mu} \right). \quad (5)$$

For $r_2/r_1 > 15.58$ the three-burn transfer beats Hohmann for sufficiently large r_i ; the unit tests check both this regime and the degenerate case $r_i = r_2$.

2.4 Elliptic transfer on a common apse line

Burns occur at true anomalies θ_1 on orbit 1 and θ_2 on orbit 2, at radii $r_A = p_1/(1 + e_1 \cos \theta_1)$ and $r_B = p_2/(1 + e_2 \cos \theta_2)$. The transfer conic shares the apse line, so $p_t = r_A(1 + e_t \cos \theta_1) = r_B(1 + e_t \cos \theta_2)$, giving

$$e_t = \frac{r_B - r_A}{r_A \cos \theta_1 - r_B \cos \theta_2}, \quad p_t = \frac{r_A r_B (\cos \theta_1 - \cos \theta_2)}{r_A \cos \theta_1 - r_B \cos \theta_2}. \quad (6)$$

With $h = \sqrt{\mu p}$, the transverse and radial velocity components on a conic are $v_\perp = h/r$ and $v_r = (\mu/h) e \sin \theta$, and each burn is

$$\Delta v = \sqrt{(v_{\perp,t} - v_\perp)^2 + (v_{r,t} - v_r)^2}. \quad (7)$$

The flight time follows from Kepler's equation between θ_1 and θ_2 on the transfer ellipse.

2.5 Single-impulse transfer with a rotated apse line

For coplanar ellipses whose apse lines differ by η , the intersection satisfies $p_1/(1 + e_1 \cos \theta) = p_2/(1 + e_2 \cos(\theta - \eta))$, i.e. $a \cos \theta + b \sin \theta = c$ with

$$a = e_1 p_2 - e_2 p_1 \cos \eta, \quad b = -e_2 p_1 \sin \eta, \quad c = p_1 - p_2. \quad (8)$$

Writing $\alpha = \text{atan2}(b, a)$, the roots are $\theta = \alpha \pm \arccos(\frac{c}{a} \cos \alpha)$ (no intersection if the argument exceeds 1 in magnitude). At the intersection radius r the velocity magnitudes $v_k = \sqrt{v_{\perp,k}^2 + v_{r,k}^2}$ and flight-path angles $\phi_k = \text{atan2}(v_{r,k}, v_{\perp,k})$ give

$$\Delta v = \sqrt{v_1^2 + v_2^2 - 2v_1 v_2 \cos(\phi_2 - \phi_1)}. \quad (9)$$

2.6 Two-impulse plane-change transfer

Problem. Find (ν_1, ν_2) and a connecting conic from $\mathbf{r}_1(\nu_1)$ on orbit 1 to $\mathbf{r}_2(\nu_2)$ on orbit 2 minimizing

$$\Delta v = \|\mathbf{v}_1 - \mathbf{v}_{c1}\| + \|\mathbf{v}_{c2} - \mathbf{v}_2\|, \quad (10)$$

where $\mathbf{v}_{c1}, \mathbf{v}_{c2}$ are the orbital velocities at the burn points and $\mathbf{v}_1, \mathbf{v}_2$ the transfer-conic velocities there. Time of flight is free.

Parametrization by the semi-latus rectum. For fixed end points with transfer angle $\Delta\theta$, the connecting conics form a one-parameter family labelled by p . The Lagrange coefficients

$$f = 1 - \frac{r_2}{p}(1 - \cos \Delta\theta), \quad g = \frac{r_1 r_2 \sin \Delta\theta}{\sqrt{\mu p}}, \quad \dot{g} = 1 - \frac{r_1}{p}(1 - \cos \Delta\theta) \quad (11)$$

give the terminal velocities in closed form,

$$\mathbf{v}_1 = \frac{\mathbf{r}_2 - f \mathbf{r}_1}{g}, \quad \mathbf{v}_2 = \frac{\dot{g} \mathbf{r}_2 - \mathbf{r}_1}{g}, \quad (12)$$

so $\Delta v(\nu_1, \nu_2, p)$ is an explicit function; no Kepler or Lambert iteration is needed [3, 4].

Bounds on p . With $k = 2r_1 r_2 \sin^2(\Delta\theta/2)$, $l = r_1 + r_2$ and $w = 2\sqrt{r_1 r_2} \cos(\Delta\theta/2)$ let $p_a = k/(l + w)$ and $p_b = k/(l - w)$. For the short way ($\Delta\theta < \pi$) every $p \in (p_a, \infty)$ is valid: ellipses below p_b , the connecting parabola at p_b , hyperbolas above. For the long way ($\Delta\theta > \pi$) the valid range is $p \in (0, p_a)$, all ellipses, with the parabola at the upper limit.

Time of flight (an output). With $e \cos \nu_1 = p/r_1 - 1$ and $e \sin \nu_1 = v_{r1} h / \mu$, $h = \sqrt{\mu p}$, the flight time between ν_1 and $\nu_2 = \nu_1 + \Delta\theta$ comes from Kepler's equation: $M = E - e \sin E$ for $e < 1$, Barker's equation for $e = 1$, and $N = e \sinh H - H$ for $e > 1$. M -revolution solutions are the same conic with M extra periods of coasting and the same Δv , so revolutions never enter the free-time search.

Reduced objective. Eliminating p and the choice of way (short/long) gives a two-variable function on the torus $[0, 2\pi)^2$:

$$F(\nu_1, \nu_2) = \min_{\text{way} \in \{\text{short}, \text{long}\}} \min_p \Delta v(\nu_1, \nu_2, p). \quad (13)$$

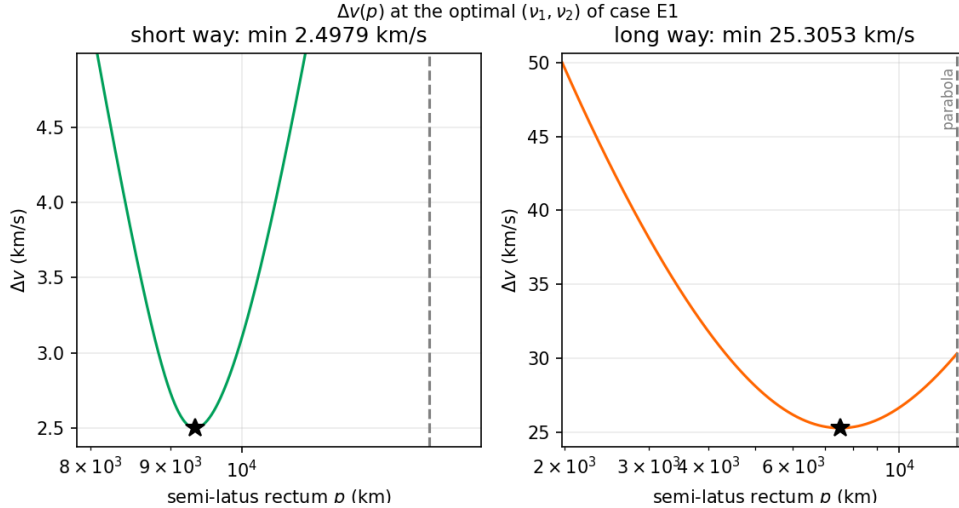


Figure 1: Δv as a function of p at the optimal burn points of the elliptic example E1, short way (left) and long way (right); the star marks the minimum and the dashed line the connecting parabola. Each curve shows a single interior minimum, consistent with the empirical unimodality used by the inner minimizer. At these burn points the long way costs about 25 km/s, so the short way (2.4979 km/s) is selected; the solver evaluates both because the better way can change from one (ν_1, ν_2) to the next.

The nodal slice ($\Delta\theta = \pi$). If the burns are at opposite ends of the line of nodes $\hat{\mathbf{d}} = \hat{\mathbf{n}}_1 \times \hat{\mathbf{n}}_2 / \|\hat{\mathbf{n}}_1 \times \hat{\mathbf{n}}_2\|$, then $\mathbf{r}_2 \parallel -\mathbf{r}_1$, $g \rightarrow 0$, and the (ν_1, ν_2, p) form is singular. Directly from the orbit equation, $1/r_1 + 1/r_2 = 2/p$, so $p = 2r_1r_2/(r_1 + r_2)$ is fixed and

$$e \cos \nu_1 = c_0 = \frac{r_2 - r_1}{r_1 + r_2}, \quad e \sin \nu_1 = q \text{ (free)}, \quad e = \sqrt{c_0^2 + q^2}. \quad (14)$$

A second free variable $\varphi \in [0, 2\pi)$ orients the transfer plane about the line of nodes. With $\hat{\mathbf{t}}(\varphi) \perp \hat{\mathbf{r}}_1$ a unit vector,

$$\mathbf{v}_1 = \sqrt{\frac{\mu}{p}} q \hat{\mathbf{r}}_1 + \frac{\sqrt{\mu p}}{r_1} \hat{\mathbf{t}}, \quad \mathbf{v}_2 = \sqrt{\frac{\mu}{p}} q \hat{\mathbf{r}}_1 - \frac{\sqrt{\mu p}}{r_2} \hat{\mathbf{t}}. \quad (15)$$

For fixed φ each term of Δv is the norm of a function affine in q , so Δv is *convex in q* : the inner minimization is guaranteed unimodal. Ellipses require $|q| < q_{\max} = 2\sqrt{r_1 r_2}/(r_1 + r_2)$; hyperbolic transfers exist only for $q < 0$.

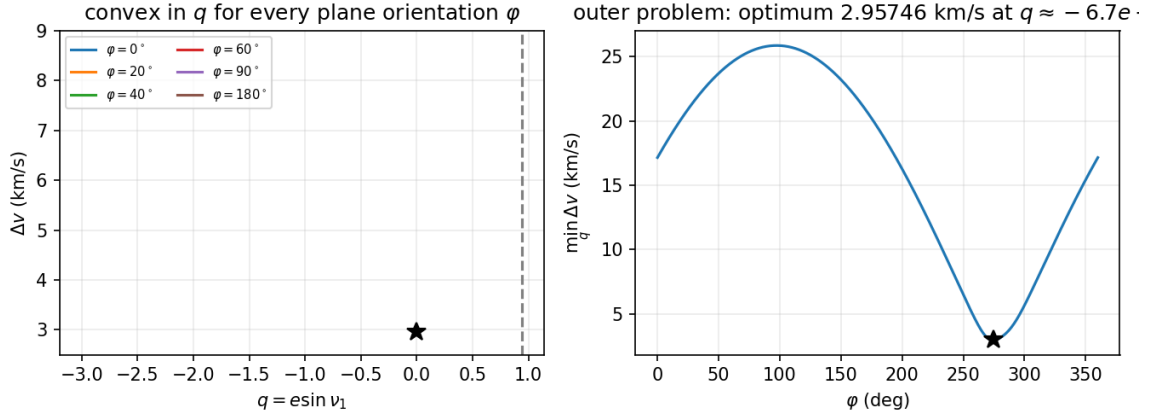


Figure 2: Nodal slice for the circular 7000 → 14000 km, $\Delta i = 20^\circ$ problem. Left: $\Delta v(q)$ for several plane orientations φ ; every curve is convex and the dashed line is the parabolic limit $q_{\max}$. Right: the outer one-dimensional problem $\min_q \Delta v$ versus φ ; the star is the optimum, 2.957460 km/s.

Closed-form checks. A pure plane change of Δi at radius r costs $\Delta v = 2\sqrt{\mu/r} \sin(\Delta i/2)$. A coplanar circular transfer reduces to the Hohmann cost.

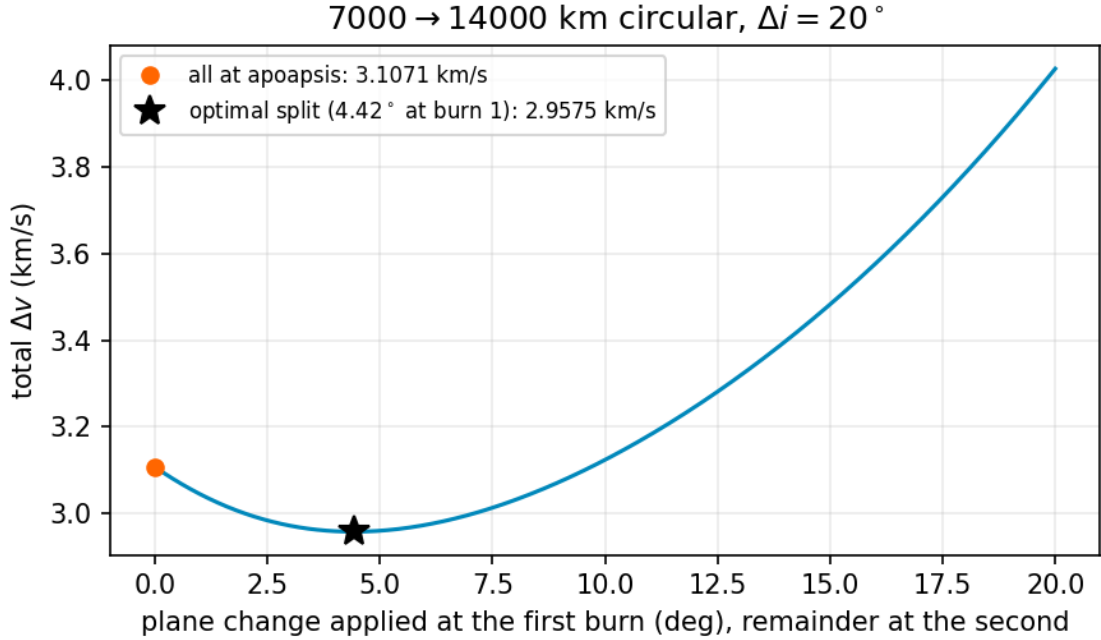


Figure 3: Hohmann transfer 7000 $\rightarrow$ 14000 km with a 20° plane change: total Δv versus how much of the plane change is made at the first burn. Putting all of it at apoapsis costs 3.1071 km/s; the optimal split (about 4.4° at the first burn) costs 2.9575 km/s, matching the solver.

3 Design Decisions

3.1 Free time and the p -parametrization instead of Lambert

The first design sampled time of flight on a grid, called an Izzo Lambert solver [1] (with Gooding-style velocity reconstruction [2]) at each point and refined. This has three drawbacks for a free-time problem: the time of flight has no natural scale; each evaluation needs an iterative solve; and $\Delta v(\Delta T)$ has several local minima (one family per revolution count, two branches per $M \geq 1$) so a coarse sample plus local refinement can land in the wrong basin. Labelling the connecting conics by p gives analytic bounds, closed-form velocities and no revolution enumeration. In a per-call micro-benchmark (CPython 3.12, NumPy 2.4) the closed-form velocities took about 40 μ s versus about 187 μ s for the Izzo solve ($\approx 4.7\times$); no end-to-end comparison was measured because the original ΔT -grid search never ran to completion. The Lambert solver is kept (Python only) as an independent cross-check: the p -family velocities agree with it to about 5×10^{-12} km/s over 180 cases.

3.2 Removing Pyodide and porting to native JavaScript

The Python solver ran in the browser through Pyodide [20] (NumPy only). It was removed for four reasons.

- **Speed.** A solve took roughly 2–6 s in native CPython with a 12–16 point grid, and about 7 s per solve under Pyodide (figure recorded in the JS test suite), which is too slow for an interactive calculator. The JS solver uses a 72×72 grid, and the test suite asserts a solve

completes in under 3 s. In a sandbox run (Node 22) the six benchmark cases solved in roughly 100–140 ms each, with the high end at 169 ms. This is about two orders of magnitude faster than the Pyodide figure.

- **Startup cost.** Pyodide plus NumPy is a multi-megabyte runtime that must be downloaded and initialized before the first plane-change solve; native JS has no such cost.
- **Complexity and robustness.** The Pyodide path needed a lazy loader, a virtual file system into which the Python modules were fetched one by one, and a JSON bridge, and it was never verified in a browser. Removing it leaves a static site with one ES-module entry point.
- **Room to improve the algorithm.** The port was not a transliteration. The JS search evaluates *both* the short and long way at every (ν_1, ν_2) , uses a 72×72 grid, and treats every grid local minimum (8-neighbourhood, periodic) as a basin seed. The best eight seeds are polished with a two-dimensional Nelder–Mead search, de-duplicated, and reported sorted by Δv . Cells within about 10^{-4} rad of collinear burns are skipped by the general search, and `computePlaneChange` separately solves the nodal slice exactly and keeps whichever result is cheaper. The Python search kept only the way that is prograde about orbit 1’s normal, which makes F jump by about 1.3 km/s across the branch flip (covered by a regression test).

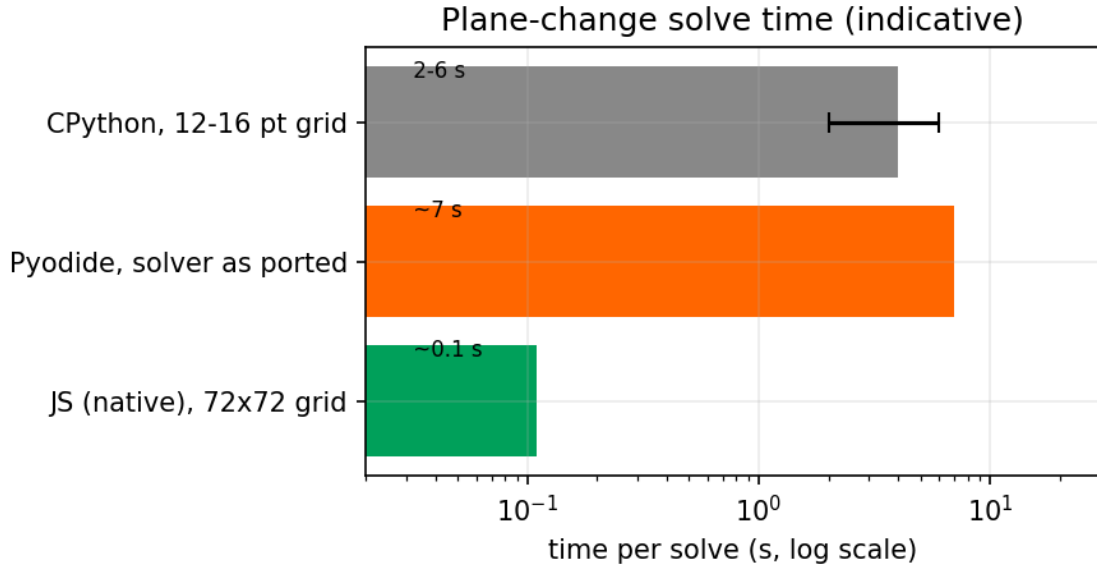


Figure 4: Indicative time per plane-change solve. The Pyodide figure is the one recorded in the JS test suite, the CPython range is from the original README, and the JS value is a sandbox measurement; none of these is a controlled benchmark.

The Python package is retained for three reasons: it generates the reference fixtures that the JS tests compare against; it is an independent implementation for cross-checking; and it contains the Izzo Lambert solver, which has no JS port yet and would be needed for a fixed-time-of-flight mode.

3.3 Choice of optimizer: Nelder–Mead versus CMA-ES versus Lipschitz branch and bound

Shape of the problem. After eliminating p with a robust 1-D minimizer (a coarse sweep to bracket, then Brent’s method [13]), the outer problem is two-dimensional, periodic, cheap to evaluate (microseconds per point), multimodal, and only piecewise smooth: F is a minimum over two branches and over p , so it can have kinks where the active way switches. It also has a singular region where $\mathbf{r}_1 \parallel \mathbf{r}_2$ ($\sin \Delta\theta \rightarrow 0$) and Δv is unbounded.

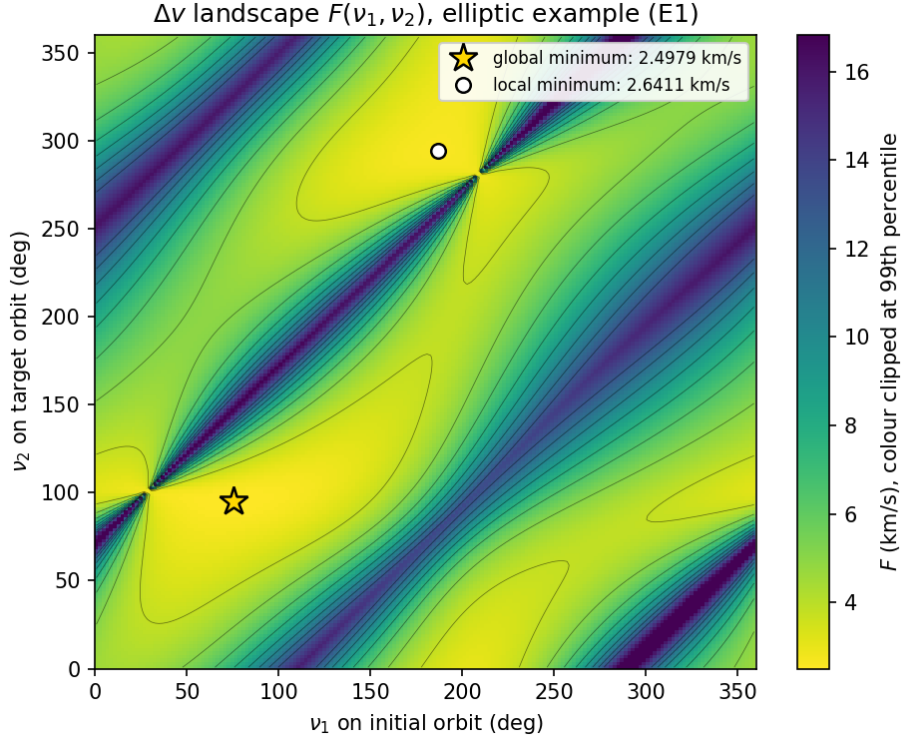


Figure 5: The reduced objective $F(\nu_1, \nu_2)$ for the elliptic example E1, computed on a 180×180 grid. The dark ridge is the region of nearly collinear burn points, where Δv is very large and the p -parametrization is singular. The star is the global minimum (2.4979 km/s) and the circle a second local minimum (2.6411 km/s), both found by the grid scan followed by Nelder–Mead polishing; colours are clipped at the 99th percentile.

Chosen approach. A fixed 72×72 grid on the torus locates the basins; Nelder–Mead [10] then polishes starting points from the best grid cells; the nodal slice is solved exactly with a convex inner problem. This is a derivative-free local method embedded in a deterministic global scan.

Why Nelder–Mead.

1. With p eliminated the dimension is only two, so the grid already supplies global exploration. What remains is high-precision local convergence, which is what a simplex method does well; CMA-ES’s strength (adapting a search distribution in higher-dimensional rugged landscapes) is not needed.

	Nelder–Mead (chosen)	CMA-ES [14, 15]	Lipschitz branch & bound [16, 17, 18, 19]
Global guarantee	None by itself; global behaviour comes from the grid and multiple starts	Probabilistic; large populations/restarts help on multimodal problems	Certified ε -optimal if a valid Lipschitz constant is known
Needs gradients	No	No	No (needs L)
Handles kinks	Yes in practice (no smoothness assumed)	Yes	Yes, but a larger L slows it
Cost for $n = 2$	Tens to hundreds of evaluations to polish	Typically hundreds to thousands, plus covariance updates	Grows rapidly as ε shrinks; tight tolerances are expensive
Deterministic	Yes	No (needs fixed seed)	Yes
Implementation in dependency-free JS	About 40 lines	Substantial (eigendecomposition, step-size control, restarts)	Moderate; the hard part is L
Main weakness here	Can stall at non-stationary points [12]; convergence theory is limited even for $n = 2$ [11]	Designed for harder, higher-dimensional problems; overhead is not justified for $n = 2$	No finite global L : $\Delta v \rightarrow \infty$ as $\sin \Delta\theta \rightarrow 0$

Table 1: Comparison of the three optimizers considered for the outer problem. The comparison is a reasoned one; CMA-ES and branch and bound were not implemented or benchmarked in this project.

2. It needs no derivatives and no smoothness, which matters for a minimum-of-branches objective. Analytic gradients of F would require differentiating through the inner minimization over p .
3. It is deterministic, so results are reproducible and unit tests can use tight tolerances.
4. It is small enough to write dependency-free in JavaScript, consistent with the static-hosting goal.

Why not Lipschitz branch and bound. A Lipschitz method is attractive because it can certify the optimum, and in two dimensions it is not hopeless in cost. The obstacle is the constant: Δv diverges as the transfer angle approaches 0 or π , so no global finite L exists; a usable L exists only after excluding neighbourhoods of the singular set, and the nodal slice $\Delta\theta = \pi$ (where optima of circular-to-circular problems actually lie) is exactly the excluded region. Even then, converging to 10^{-9} km/s costs far more evaluations than Nelder–Mead polishing. A hybrid, using branch and bound away from the singular set to certify the grid result and Nelder–Mead to polish, is a natural extension. Likewise, Lawden’s primer-vector conditions [8] would certify whether a third impulse could lower the cost, and the coplanar bi-impulse literature [5] optimizes over the same variables; neither is implemented.

Why not CMA-ES. For $n = 2$ it would work, but it is stochastic, needs restarts to be reliably global, spends many evaluations learning a covariance matrix that a two-dimensional grid makes unnecessary, and is a much larger piece of code to port and test.

Evidence that the combination is adequate (not a proof). (i) The answer is independent of grid size (36, 72, 144 points agree to 10^{-6} km/s on two elliptic test cases); (ii) the JS result is never worse than a high-effort Python search (48×48 grid, 12 polish starts) on six cases, beating it where the Python values contain rounding-noise artefacts near the nodal line; (iii) the nodal solver and general search agree on the circular cases; and (iv) the basins found are reported, sorted by Δv , so a user can see when several local minima exist. Nelder–Mead can in principle converge to a non-minimum, and a grid can miss a very narrow basin, so global optimality is established empirically rather than proved.

3.4 Safeguards in the solver

- **Both ways at every grid point.** Short and long ways are both evaluated and the better kept, removing the discontinuity of a one-sense search.
- **Degenerate cells.** Cells within about 10^{-4} rad of $\mathbf{r}_1 \parallel \mathbf{r}_2$ are rejected from the general search because the $1/\sin \Delta\theta$ factor amplifies rounding error until the optimizer exploits noise; the nodal solver owns that region.
- **Open-interval mapping for p .** p is mapped from an open interval to $\mathbb{R}$ (scaled by the gap to the parabola for the short way) so searches can never leave the valid range.
- **Result self-checks.** The returned conic passes through both burn points, the two burns sum to the total, and the minimum radius on the *travelled arc* (not the whole conic) is compared with the body radius; the UI warns if the transfer passes inside the planet.

3.5 Known limitations and future work

- **Two impulses are not always optimal.** For large plane changes or large radius ratios a three-impulse solution can be cheaper. Lawden’s primer-vector conditions [8] would certify optimality or show when a mid-course impulse helps; this is not implemented.
- **No global guarantee.** The grid-plus-Nelder–Mead search can in principle miss a very narrow basin. The solver reports every basin it finds, and global optimality is established empirically, not proved.
- **Free time only.** There is no maximum or fixed time-of-flight option. A maximum time would tighten the bound on p ; a fixed time needs the Lambert solver, which currently exists only in the Python reference.
- **Near-collinear burn points** (within about 10^{-4} rad) are excluded from the general search and covered only by the nodal slice, which is exact for burns at opposite ends of the line of nodes.
- **Flat optima.** When the optimum is a continuous family (for example identical orbits), the basin list can contain many equivalent entries.
- **Planet radius** is checked against the transfer arc only for plane change; the other transfer types use it for drawing only. The apse-line-rotation transfer is a single-impulse intersection and its plot does not mark the burn point.

3.6 Code organization

Math modules are pure functions with no DOM access and are unit-tested in Node; one schema per transfer type renders the inputs and reads them back; one plotter per transfer type draws the Desmos expressions; and the plane-change solver is split into geometry, p -family evaluation, optimizers, search, nodal solver and Kepler utilities.

4 Features and Images

4.1 Feature list

- Five transfer types selectable from one menu; the form re-renders from a per-type schema.
- Central body defined by mass and radius ($\mu = GM$ computed in the app).
- Results box: total and per-burn Δv , burn true anomalies, transfer semi-latus rectum, transfer time, and for plane change the search method (nodal or general).
- 3-D Desmos plot with colour-coded legend (initial orbit, target orbit, transfer arc, planet); the plot stays in view while the form scrolls.
- Input validation before computation, with error messages instead of NaN output.
- Warning when the transfer arc passes inside the central body.
- Static, serverless deployment; responsive layout with a “mission control” theme.

4.2 Screenshots

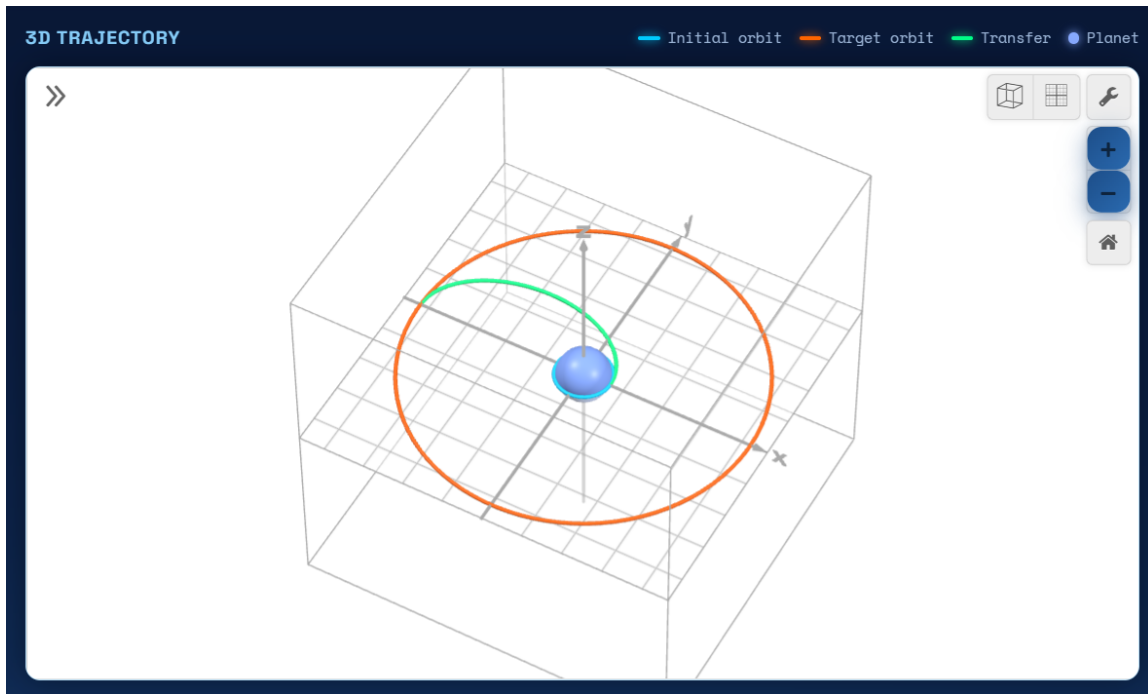


Figure 6: Hohmann transfer from LEO to GEO ($r_1 = 6678$ km, $r_2 = 42164$ km).

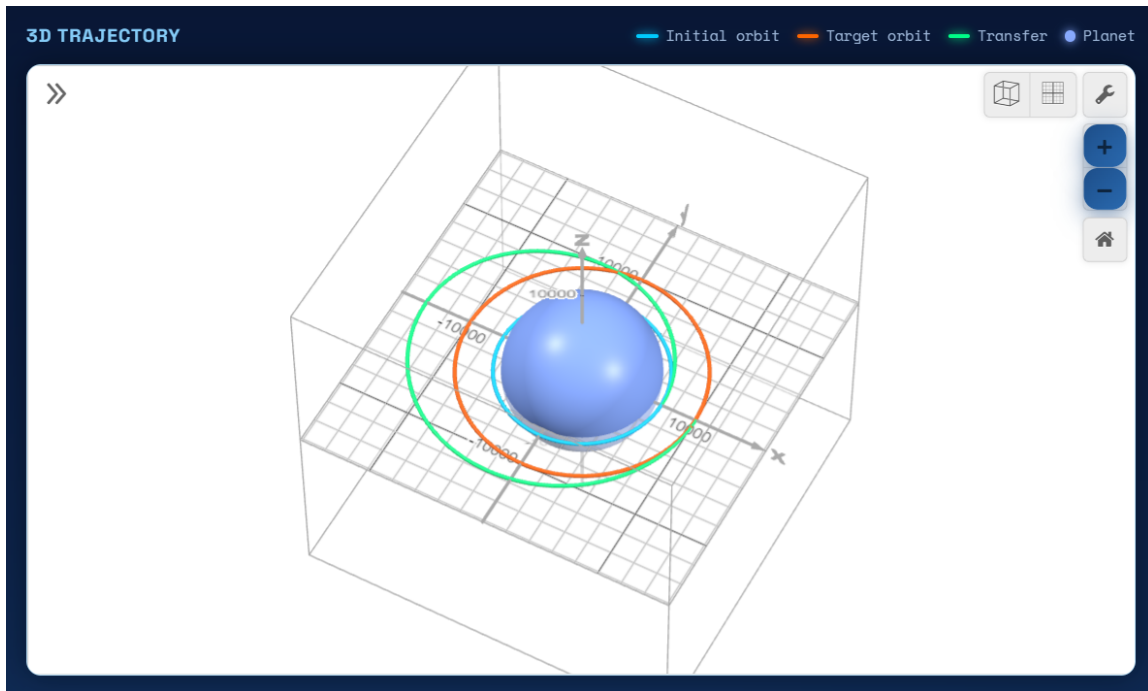


Figure 7: Bi-elliptic transfer with intermediate apoapsis r_i .

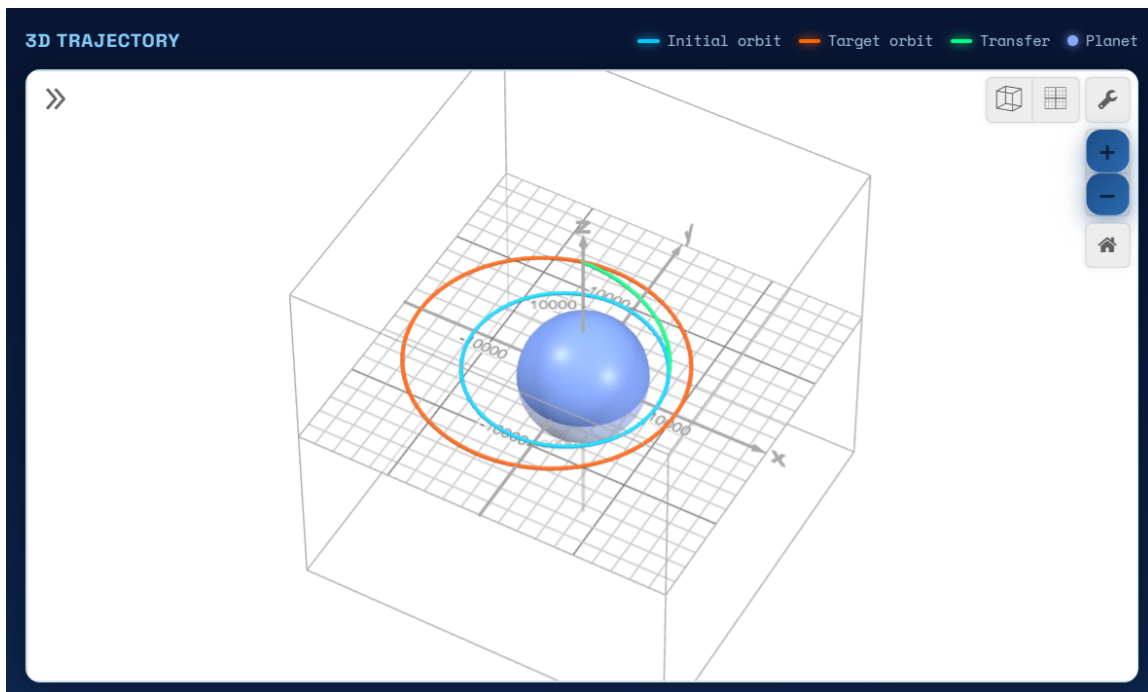


Figure 8: Elliptic transfer on a common apse line.

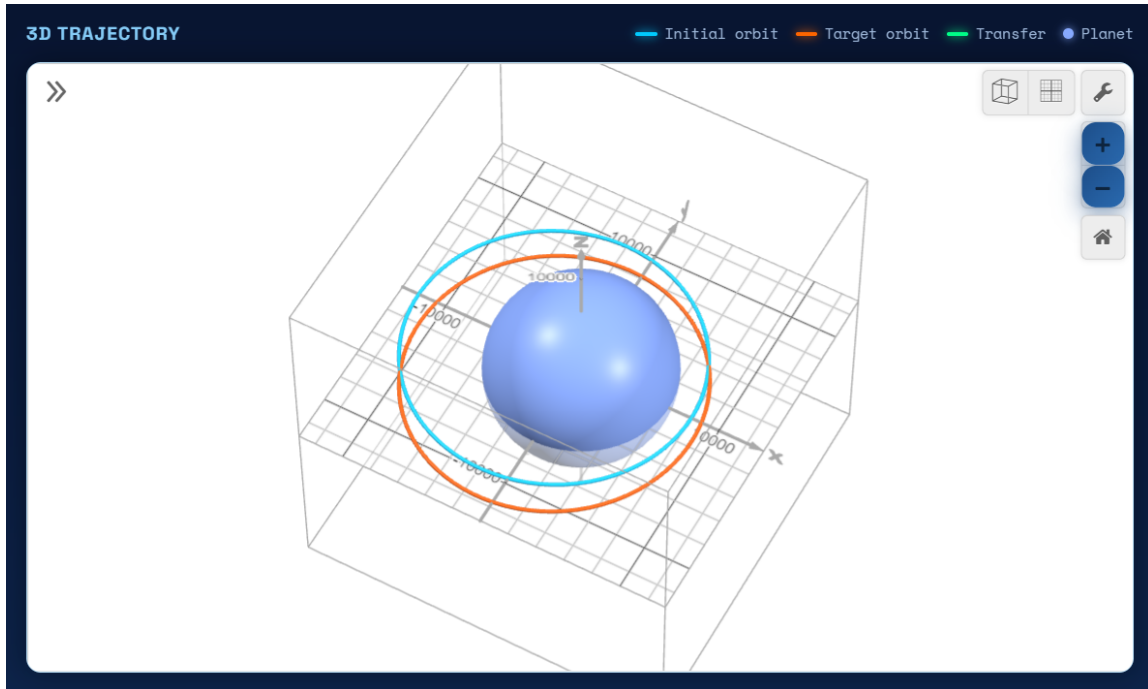


Figure 9: Coplanar ellipses with a rotated apse line.

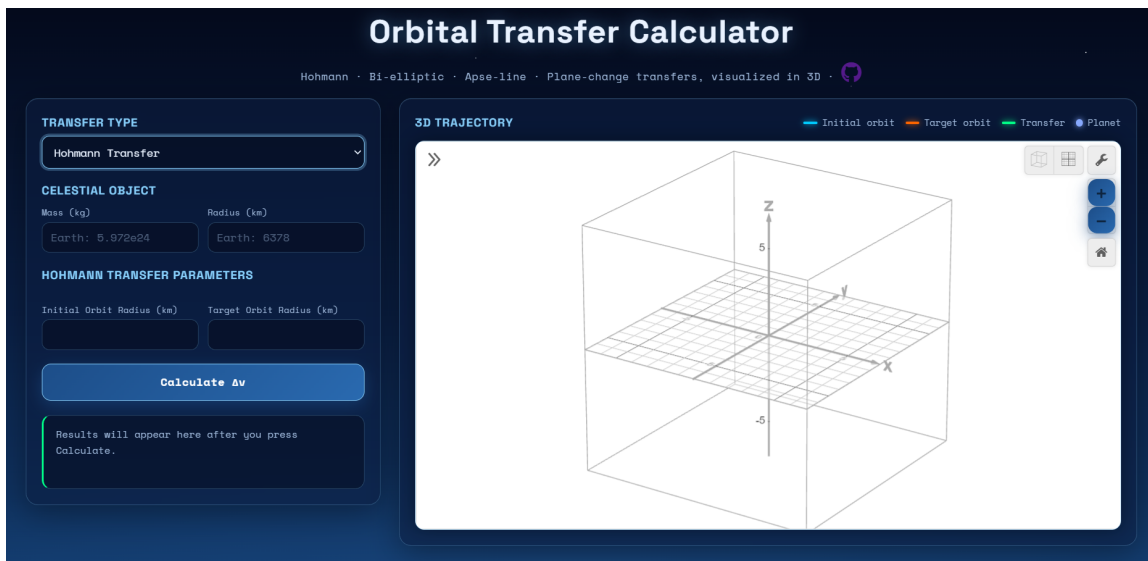


Figure 10: Full application window: input form, results box and 3-D plot.

4.3 Rendered trajectories

Figure 11 shows the solver output for two plane-change problems, rendered with matplotlib (not Desmos screenshots): burn points are marked with a circle (first burn) and a square (second burn).

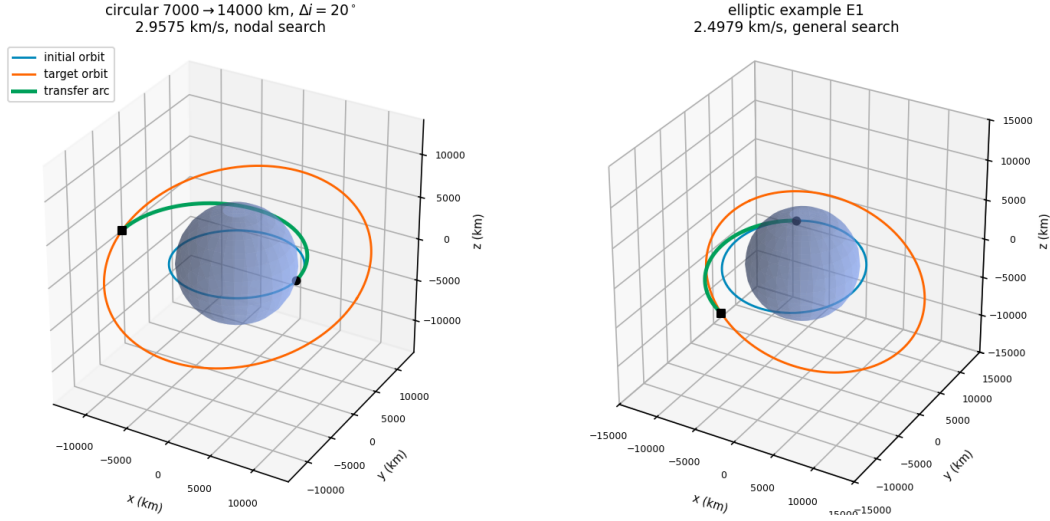


Figure 11: Optimal two-impulse transfers (green) between the initial (blue) and target (orange) orbits. Left: circular 7000 → 14000 km with $\Delta i = 20^\circ$, solved by the nodal search. Right: elliptic example E1, solved by the general search.

4.4 Verification summary

Check	Result
Coplanar circular 7000 → 14000 km	Hohmann $\Delta v = 2.14653$ km/s, $a = 10500$ km, $e = 1/3$
Pure 20° plane change at 7000 km	$2v \sin(\Delta i/2) = 2.62072$ km/s, exact to six digits
7000 → 14000 km with a 20° plane change	2.9575 km/s vs. 3.1071 km/s for Hohmann with the whole plane change at apoapsis (4.8% less)
LEO to GEO with 28.5° plane change	4.1209 km/s (Python general search)
p -family vs. Izzo Lambert	velocity agreement to about 5×10^{-12} km/s over 180 cases
Izzo solutions vs. independent propagator	endpoint error about 3×10^{-9} km over 91 solutions
Worked example of [3] ($r_1 = 9000$, $r_2 = 15000$ km, $\Delta\theta = 120^\circ$)	p bounds $0.6317r_1$ and $1.8173r_1$; minimum- Δv orbit at $p = 1.3128r_1$ gives $0.1563 v_c$ and $0.4607 T_c$

Table 2: Selected verification results from the Python reference suite and the JS test suite.

References

- [1] D. Izzo, “Revisiting Lambert’s problem,” *Celestial Mechanics and Dynamical Astronomy* **121**, 1–15 (2015). doi:10.1007/s10569-014-9587-y.
- [2] R. H. Gooding, “A procedure for the solution of Lambert’s orbital boundary-value problem,” *Celestial Mechanics and Dynamical Astronomy* **48**, 145–165 (1990).
- [3] P. R. Blanco, “Connect the dots... finding all possible orbits between two points,” *European Journal of Physics* **46**, 045004 (2025). doi:10.1088/1361-6404/ade37d; arXiv:2508.02695.

- [4] K. G. Eckel, N. X. Vinh “Optimal switching conditions for minimum fuel fixed time transfer between non coplanar elliptical orbits,” *Acta Astronautica* **11**(10–11), 621–631 (1984).
- [5] M. Sanatifar, R. Capuzzo-Dolcetta “Optimal bi-impulse orbital transfer between coplanar orbits,” EUCASS 2017-151.
- [6] H. D. Curtis, *Orbital Mechanics for Engineering Students*, Butterworth-Heinemann/Elsevier.
- [7] R. R. Bate, D. D. Mueller, J. E. White, *Fundamentals of Astrodynamics*, Dover (1971).
- [8] D. F. Lawden, *Optimal Trajectories for Space Navigation*, Butterworths (1963).
- [9] W. Hohmann, *Die Erreichbarkeit der Himmelskörper*, Oldenbourg (1925).
- [10] J. A. Nelder and R. Mead, “A simplex method for function minimization,” *The Computer Journal* **7**(4), 308–313 (1965).
- [11] J. C. Lagarias, J. A. Reeds, M. H. Wright, P. E. Wright, “Convergence properties of the Nelder–Mead simplex method in low dimensions,” *SIAM J. Optimization* **9**(1), 112–147 (1998).
- [12] K. I. M. McKinnon, “Convergence of the Nelder–Mead simplex method to a nonstationary point,” *SIAM J. Optimization* **9**(1), 148–158 (1998).
- [13] R. P. Brent, *Algorithms for Minimization without Derivatives*, Prentice-Hall (1973).
- [14] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation* **9**(2), 159–195 (2001).
- [15] N. Hansen, “The CMA evolution strategy: a tutorial,” arXiv:1604.00772 (2016).
- [16] S. A. Piyavskii, “An algorithm for finding the absolute extremum of a function,” *USSR Computational Mathematics and Mathematical Physics* **12**(4), 57–67 (1972).
- [17] B. O. Shubert, “A sequential method seeking the global maximum of a function,” *SIAM J. Numerical Analysis* **9**(3), 379–388 (1972).
- [18] D. R. Jones, C. D. Perttunen, B. E. Stuckman, “Lipschitzian optimization without the Lipschitz constant,” *J. Optimization Theory and Applications* **79**(1), 157–181 (1993).
- [19] C. Malherbe and N. Vayatis, “Global optimization of Lipschitz functions,” *Proc. ICML*, PMLR **70**, 2314–2323 (2017).
- [20] Pyodide Development Team, *Pyodide: Python distribution for the browser and Node.js based on WebAssembly*, <https://pyodide.org>.
- [21] Desmos Studio, *Desmos 3D Calculator API*, <https://www.desmos.com/api>.
- [22] Project repo: <https://github.com/Sagittarius-B-Astro/OrbitalTransferCalculator>.